

Analisi dell'Oscillatore Armonico Forzato e Smorzato tramite Metodo Runge-Kutta 4

Autore:

Alberto Demarco

Sotto la guida di

Prof. Giuseppe Magnifico

Dipartimento Interuniversitario di Fisica "Michelangelo
Merlin"

Università degli Studi di Bari "Aldo Moro"

A.A. 2025/2026



Indice

1	Introduzione	1
2	Metodologia e Codice	1
2.1	Gestione del Transitorio	1
3	Risultati e Discussione	2
3.1	Oscillatore smorzato senza forzante	2
3.2	Regime Stazionario	2
3.3	Curva di Risonanza	3
3.4	Spazio delle Fasi	3
4	Conclusioni	4
5	Appendice: Codice Python	5

Abstract

L'obiettivo di questa esperienza è l'implementazione in Python e lo studio del metodo numerico Runge-Kutta del 4° ordine (RK4), applicato all'integrazione delle equazioni del moto per un oscillatore armonico smorzato e forzato. Vengono analizzati il decadimento nel tempo, la conservazione e dissipazione dell'energia, il bilancio energetico e il fenomeno della risonanza al variare della frequenza della forzante. Si dimostra inoltre l'efficacia dell'algoritmo nel discriminare il regime transitorio da quello stazionario attraverso l'analisi dello spazio delle fasi.

1 Introduzione

Consideriamo una massa m collegata a una molla di costante elastica k , soggetta a:

- Forza elastica: $F_{el} = -kx$
- Forza di attrito viscoso: $F_{attr} = -bv$
- Forza esterna periodica: $F_{est} = F_0 \cos(\omega t)$

Dividendo per la massa m e introducendo la frequenza naturale $\omega_0 = \sqrt{k/m}$, il coefficiente di smorzamento $\gamma = b/m$ e l'ampiezza della forzante $A = F_0/m$, otteniamo l'equazione del moto in forma standard:

$$\frac{d^2x}{dt^2} = -\omega_0^2 x - \gamma \frac{dx}{dt} + A \cos(\omega t) \quad (1)$$

Introducendo la velocità $v = dx/dt$, trasformiamo l'equazione in un sistema di due equazioni differenziali ordinarie (ODE) del primo ordine:

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv}{dt} = -\omega_0^2 x - \gamma v + A \cos(\omega t) \end{cases} \quad (2)$$

A differenza del pendolo semplice, in questo sistema la forzante dipende

esplicitamente dal tempo t e vi è una dipendenza dalla velocità che rende impossibile l'uso di metodi simplettici come il Velocity-Verlet.

2 Metodologia e Codice

Per risolvere il sistema si è adottato il metodo di Runge-Kutta al quarto ordine (RK4). Poiché la forzante dipende da t , è fondamentale valutarla ai tempi corretti nei passi intermedi dell'algoritmo. Nel codice Python implementato, i coefficienti k_2 e k_3 vengono calcolati valutando esplicitamente la forzante al tempo $t_i + h/2$, mentre per k_4 al tempo $t_i + h$.

2.1 Gestione del Transitorio

Una delle idee implementative risiede nell'isolamento del regime stazionario. Dopo un transitorio iniziale, il sistema raggiunge un regime stazionario in cui oscilla alla frequenza della forzante ω . Invece di stimare empiricamente e visivamente la durata del transitorio, si è sfruttato il tempo caratteristico di decadimento $\tau = 2/\gamma$. Il codice esegue un taglio dinamico (*slicing*) degli array troncando i primi 5τ secondi della simulazione:

```
1 tau = (2 / gamma)
2 i_trans = min(int(5 * tau / h),
               int(len(t) * 2 / 3))
```

In questo modo, l'ampiezza numerica a regime viene calcolata tramite la funzione `np.max(np.abs(x[i_trans:]))`, evitando l'interferenza delle ampiezze spurie iniziali.

3 Risultati e Discussione

3.1 Oscillatore smorzato senza forzante

Nell'Esercizio 1 si pone $A = 0$. Partendo da una posizione iniziale $x_0 = 1$, la simulazione mostra il progressivo smorzamento dell'ampiezza di oscillazione nel tempo.

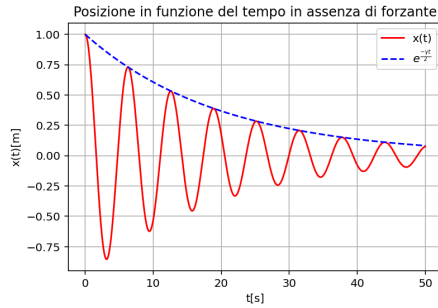


Figura 1: Esercizio 1: Posizione in funzione del tempo. Si osserva l'involuppo esponenziale.

Come si evince dalla Figura 1, l'algoritmo numerico segue perfettamente l'involuppo teorico, verificando che l'ampiezza decade esponenzialmente come $\exp(-\gamma t/2)$. A differenza di sistemi ideali, qui l'energia non si conserva a causa della dissipazione per attrito. La Figura 2 mostra l'energia meccanica totale decrescere in accordo con la teoria.

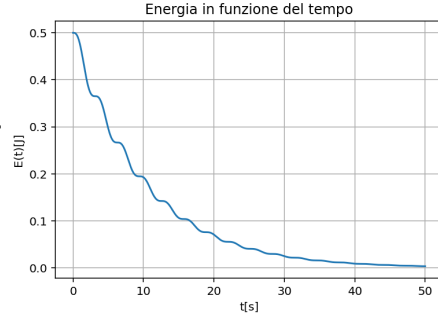


Figura 2: Esercizio 1: Decadimento dell'energia nel tempo per $\gamma = 0.1$.

3.2 Regime Stazionario

Introducendo la forzante ($A = 1$, $\omega = 0.8$, $\gamma = 0.2$), il sistema evidenzia un comportamento bifasico.

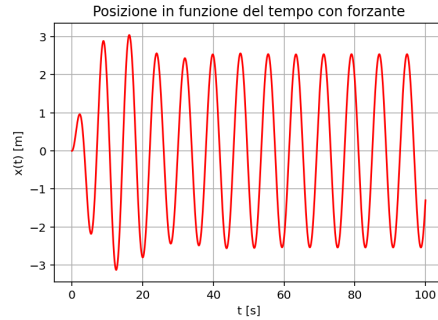


Figura 3: Esercizio 2: Evoluzione temporale con forzante. Si nota il transitorio nei primi secondi.

La soluzione stazionaria teorica prevede un'ampiezza:

$$A_s = \frac{A}{\sqrt{(\omega_0^2 - \omega^2)^2 + (\gamma\omega)^2}} \quad (3)$$

Sostituendo i parametri otteniamo un valore teorico atteso $A_s = 2.5384$. Scartando i primi 40 secondi di transitorio e analizzando il regime a regime, l'algoritmo RK4 estrae un'ampiezza numerica pari a $A_{num} = 2.5503$. L'errore relativo rispetto alla stima analitica risulta estremamente basso (0.47%), a riprova dell'accuratezza temporale dell'algoritmo.

3.3 Curva di Risonanza

Per studiare il fenomeno della risonanza, si è variata la frequenza della forzante ω nel range $[0.5, 1.5]$. Quando la frequenza della forzante si avvicina alla frequenza naturale ($\omega \approx \omega_0$), l'ampiezza delle oscillazioni diventa massima.

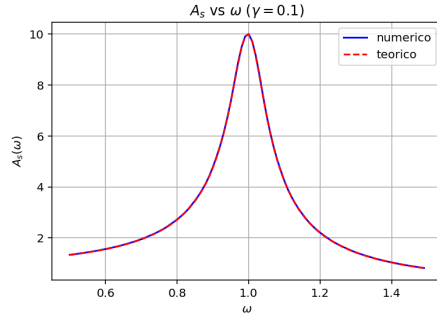


Figura 4: Esercizio 3: Curva di risonanza per $\gamma = 0.1$.

L'Esercizio 3 è stato iterato per due valori del coefficiente di smorzamento γ . L'errore relativo medio registrato sull'intero array di frequenze è di appena lo 0.37% per $\gamma = 0.1$ e cala ulteriormente allo 0.24% per $\gamma = 0.3$.

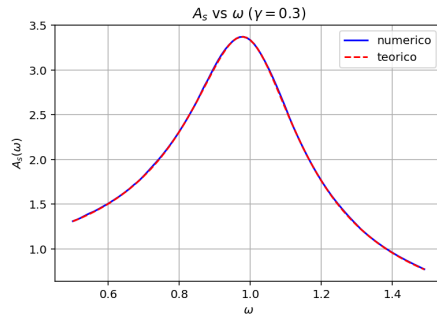


Figura 5: Esercizio 3: Curva di risonanza per $\gamma = 0.3$. Il picco si abbassa e si allarga.

Dal confronto tra Figura 4 e 5 si conferma che il fattore di qualità $Q = \omega_0/\gamma$ determina quanto è "stretto" il picco. All'aumentare dello smorzamento il picco si abbassa notevolmente, in

pieno accordo con le previsioni teoriche.

3.4 Spazio delle Fasi

L'Esercizio 4 impone l'analisi del piano delle fasi (v vs x) per discernere la topologia delle soluzioni.

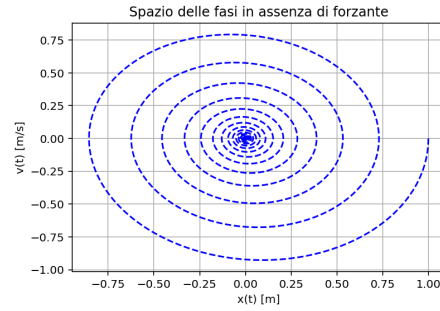


Figura 6: Esercizio 4: Spazio delle fasi (oscillatore smorzato). La traiettoria è una spirale.

In assenza di forzante, l'energia decresce monotonicamente: nello spazio delle fasi l'orbita si chiude a spirale verso l'origine (attrattore stabile), corrispondente all'equilibrio termodinamico e meccanico (Figura 6).

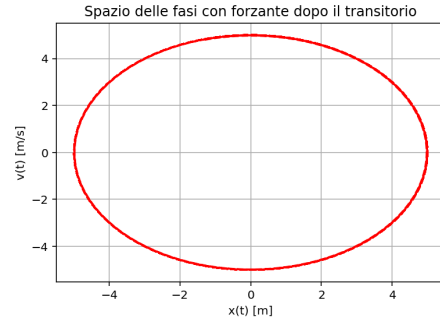


Figura 7: Esercizio 4: Spazio delle fasi (oscillatore forzato). La traiettoria si stabilizza su un ciclo limite ellittico.

L'introduzione della forzante cambia drasticamente la situazione: nel regime stazionario le due potenze (dissipata dall'attrito e immessa dalla for-

zante) si bilanciano in media. Questo equilibrio dinamico produce nello spazio delle fasi un'orbita chiusa isolata, detta *ciclo limite* (Figura 7). L'ellisse tracciata dimostra che le coordinate x e v oscillano armonicamente con una differenza di fase costante.

4 Conclusioni

L'implementazione del metodo RK4 ha permesso di simulare con elevato grado di accuratezza la dinamica di un oscillatore armonico in presenza di forzante dipendente dal tempo e smorzamento. La cura nell'implementazione, testimoniata dall'uso dei punti intermedi esatti per la forzante e dall'isolamento analitico del transitorio tramite il parametro τ , ha permesso di ricavare le curve di risonanza con un errore medio inferiore allo 0.5%. Lo studio nello spazio delle fasi ha infine confermato empiricamente la transizione da spirale a ciclo limite per i sistemi dissipativi forzati.

5 Appendice: Codice Python

Viene qui riportato il codice integrale utilizzato per le simulazioni, in cui si evidenziano le funzioni RK4, `energia` e `A_s`, oltre alla logica iterativa adottata per la misurazione degli errori e lo sfasamento dei transitori.

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  import numpy as np
4  from matplotlib import pyplot as plt
5
6  def main():
7      #Esercizio 1
8      A = 0.0
9      x_0 = 1.0
10     v_0 = 0.0
11     omega_0 = 1.0
12     omega = 0.0
13     gamma = 0.1
14     m = 1.0
15     h = 0.01
16     t_max = 50
17
18     t = np.arange(0, t_max, h)
19     x, v = RK4(t, h, x_0, v_0, omega_0, gamma, A, omega)
20     decad = np.exp(-gamma*t/2)
21     E = energia(x, v, m, omega_0)
22
23     # Plot e analisi Esercizio 1 omissi per brevit  nel testo...
24
25     #Esercizio 2
26     A = 1.0
27     x_0 = 0.0
28     v_0 = 0.0
29     omega_0 = 1.0
30     omega = 0.8
31     gamma = 0.2
32     m = 1.0
33     h = 0.01
34     t_max = 100
35
36     A_staz = A_s(A, omega_0, omega, gamma)
37     t = np.arange(0, t_max, h)
38     x, v = RK4(t, h, x_0, v_0, omega_0, gamma, A, omega)
39
40     tau = (2 / gamma)
41     i_trans = min(int(5 * tau / h), int(len(t) * 2 / 3))
42     A_num = np.max(np.abs(x[i_trans:]))
43
44     print("A_s th = %.4f" % A_staz)
45     print("A_s num = %.4f" % A_num)
46     print("Errore relativo = %.2f %" % (np.abs(A_staz - A_num)/
47         A_staz * 100))
48
49     #Esercizio 3
50     A = 1.0
51     x_0 = 0.0
52     v_0 = 0.0
53     omega_0 = 1.0
54     omega_values = np.arange(0.5, 1.5, 0.01)
55     gamma = 0.1
56     m = 1.0
```

```

56     h = 0.01
57     t_max = 200
58
59     t = np.arange(0, t_max, h)
60     A_s_values_th = np.zeros(len(omega_values))
61     A_s_values_num = np.zeros(len(omega_values))
62     epsilon = np.zeros(len(omega_values))
63
64     for i in range(len(omega_values)):
65         omega = omega_values[i]
66         A_staz_th = A_s(A, omega_0, omega, gamma)
67         A_s_values_th[i] = A_staz_th
68
69         x = RK4(t, h, x_0, v_0, omega_0, gamma, A, omega)[0]
70         tau = (2 / gamma)
71         i_trans = min(int(5 * tau / h), int(len(t) * 2 / 3))
72         A_num = np.max(np.abs(x[i_trans:]))
73         A_s_values_num[i] = A_num
74
75         errore_rel = (np.abs(A_staz_th - A_num) / A_staz_th * 100)
76         epsilon[i] = errore_rel
77
78     err_rel_medio = np.mean(epsilon)
79     print("Errore relativo medio: %.2f %" % err_rel_medio)
80
81     #Esercizio 4
82     h = 0.01
83     t_max = 150
84     t = np.arange(0, t_max, h)
85
86     x, v = RK4(t, h, x_0=1.0, v_0=0.0, omega_0=1.0, gamma=0.1, A
87               =0.0, omega=0.0)
88
89     x_forz, v_forz = RK4(t, h, x_0=0.0, v_0=0.0, omega_0=1.0, gamma
90               =0.2, A=1, omega=1.0)
91
92     tau = (2 / 0.2)
93     i_trans = min(int(5 * tau / h), int(len(t) * 2 / 3))
94
95     def RK4(t, h, x_0, v_0, omega_0, gamma, A, omega):
96         x = np.zeros(len(t))
97         v = np.zeros(len(t))
98         x[0] = x_0
99         v[0] = v_0
100
101         for i in range(len(t) - 1):
102             k_1_x = v[i]
103             k_1_v = -(omega_0**2)*x[i] - gamma*v[i] + A*np.cos(omega*t[
104                 i])
105
106             k_2_x = v[i] + (h/2)*k_1_v
107             k_2_v = -(omega_0**2)*(x[i] + (h/2)*k_1_x) - gamma*(v[i] +
108                 (h/2)*k_1_v) + A*np.cos(omega*(t[i] + (h/2)))
109
110             k_3_x = v[i] + (h/2)*k_2_v
111             k_3_v = -(omega_0**2)*(x[i] + (h/2)*k_2_x) - gamma*(v[i] +
112                 (h/2)*k_2_v) + A*np.cos(omega*(t[i] + (h/2)))
113
114             k_4_x = v[i] + h * k_3_v
115             k_4_v = -(omega_0**2)*(x[i] + h*k_3_x) - gamma*(v[i] + h*
116                 k_3_v) + A*np.cos(omega*(t[i] + h))
117
118             x[i+1] = x[i] + (h/6)*(k_1_x + 2*k_2_x + 2*k_3_x + k_4_x)

```



```

112         v[i+1] = v[i] + (h/6)*(k_1_v + 2*k_2_v + 2*k_3_v + k_4_v)
113
114     return x, v
115
116 def energia(x, v, m, omega_0):
117     E = 0.5*m*(v**2 + (omega_0*x)**2)
118     return E
119
120 def A_s(A, omega_0, omega, gamma):
121     A_staz = A/np.sqrt((omega_0**2 - omega**2)**2 + (gamma*omega)
122                       **2)
123     return A_staz
124
125 main()

```